

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА  
ВЕЛИКОГО**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Отчёт о выполнении лабораторных работ  
по дисциплине:

# Высокоскоростные сетевые технологии суперкомпьютеров

Студент: \_\_\_\_\_

Салимли Айзек Мухтар Оглы

Преподаватель: \_\_\_\_\_

Курочкин Михаил Александрович

«\_\_\_\_\_» \_\_\_\_\_ 20\_\_ г.

# Содержание

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Введение</b>                                                              | <b>3</b>  |
| <b>1 Лабораторная работа №1</b>                                              | <b>4</b>  |
| 1.1 Постановка задачи . . . . .                                              | 4         |
| 1.2 Математическое описание . . . . .                                        | 4         |
| 1.3 Особенности реализации . . . . .                                         | 5         |
| 1.4 Сборка и запуск собственного теста (CUDA) . . . . .                      | 5         |
| 1.5 Результаты эксперимента . . . . .                                        | 5         |
| 1.6 Заключение лабораторной работы №1 . . . . .                              | 11        |
| <b>2 Лабораторная работа №2</b>                                              | <b>12</b> |
| 2.1 Постановка задачи . . . . .                                              | 12        |
| 2.2 Математическое описание . . . . .                                        | 12        |
| 2.2.1 Алгоритм решения задачи вычисления алгебраических дополнений . . . . . | 12        |
| 2.3 Особенности MPI-реализации . . . . .                                     | 13        |
| 2.4 Особенности CUDA-реализации . . . . .                                    | 13        |
| 2.5 Результаты эксперимента . . . . .                                        | 13        |
| 2.6 Заключение лабораторной работы №2 . . . . .                              | 18        |
| <b>Заключение</b>                                                            | <b>19</b> |
| <b>Приложение А</b>                                                          | <b>20</b> |
| <b>Приложение Б</b>                                                          | <b>29</b> |
| <b>Приложение В</b>                                                          | <b>30</b> |
| <b>Приложение Г</b>                                                          | <b>32</b> |
| <b>Приложение Д</b>                                                          | <b>33</b> |
| <b>Приложение Е</b>                                                          | <b>34</b> |
| <b>Приложение Ж</b>                                                          | <b>38</b> |
| <b>Приложение З</b>                                                          | <b>42</b> |
| <b>Приложение И</b>                                                          | <b>43</b> |
| <b>Приложение Й</b>                                                          | <b>44</b> |

## Введение

Современные вычислительные системы опираются на несколько взаимодополняющих моделей параллелизма. Графические ускорители с архитектурой массового параллелизма позволяют задействовать тысячи нитей при обработке регулярных численных структур; в экосистеме NVIDIA для этого широко применяется технология CUDA. Для многопроцессорных и многомашинных конфигураций стандартным средством координации выступает интерфейс MPI (Message Passing Interface), обеспечивающий обмен сообщениями между процессами, в том числе при работе на вычислительном кластере.

Оценка практической производительности суперкомпьютеров и вычислительных узлов традиционно опирается на решение систем линейных алгебраических уравнений; этой цели служит тест LINPACK, результаты которого выражаются в показателях вроде пиковой и достигнутой производительности (Rmax, число операций с плавающей точкой в единицу времени).

Цель работы - изучить подходы к оценке производительности и сопоставить различные технологии параллельного программирования на примере двух задач.

В **первой лабораторной работе** рассматривается итерационное решение СЛАУ методом Якоби на GPU с использованием CUDA; полученные времена сопоставляются с запуском эталонного теста Intel LINPACK на CPU узла с графическим ускорителем. Таким образом, сочетаются исследование характерного для HPC и практика разработки CUDA-ядра для плотной линейной алгебры.

Во **второй лабораторной работе** решается прикладная задача построения матрицы алгебраических дополнений заданной квадратной матрицы. Требуется реализовать распределённый вариант на кластере средствами MPI, сопоставить его с исходной CUDA-реализацией и проанализировать зависимость времени выполнения от числа задействованных процессов и узлов.

Эксперименты проводились на ресурсах суперкомпьютерного центра «Политехнический»; для удалённого доступа использовались учётные данные, выданные для работы в СКЦ.

В качестве пользователя: **tm3u20**. Вход на кластер выполнялся через узел доступа `login1.hpc.spbstu.ru` с использованием SSH-ключа.

# 1 Лабораторная работа №1

## 1.1 Постановка задачи

В рамках первого задания требовалось исследовать производительность вычислительной системы при решении плотной системы линейных алгебраических уравнений и сопоставить результаты собственной реализации с эталонным тестом LINPACK. Для достижения этой цели были поставлены следующие задачи:

- изучить подходы к оценке производительности вычислительных систем;
- разработать собственную CUDA-реализацию LINPACK-подобного теста;
- выполнить экспериментальные запуски на вычислительном узле СКЦ Политехнический;
- сравнить результаты собственной программы и стандартного Intel LINPACK.

## 1.2 Математическое описание

Тест LINPACK применяется для оценки производительности вычислительных систем при решении плотной системы линейных уравнений. Рассматривается задача

$$Ax = b, \quad (1)$$

где  $A \in \mathbb{R}^{n \times n}$  - плотная квадратная матрица,  $x \in \mathbb{R}^n$  - искомый вектор решения,  $b \in \mathbb{R}^n$  - вектор правой части.

В классическом варианте LINPACK обычно применяются прямые методы, основанные на LU-разложении [?]. В собственной реализации для первого задания использован итерационный метод Якоби, так как он хорошо распараллеливается на GPU: каждая строка матрицы может обрабатываться независимо [?].

Для метода Якоби очередное приближение вычисляется по формуле

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left( b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n. \quad (2)$$

Критерий остановки выбран в виде

$$\max_i \left| x_i^{(k+1)} - x_i^{(k)} \right| \leq \varepsilon. \quad (3)$$

Чтобы обеспечить сходимость метода, в программе формируется строго диагонально доминирующая матрица. Для контроля качества решения дополнительно вычисляются:

$$\|Ax - b\|_\infty, \quad (4)$$

$$\|x - x_{true}\|_\infty, \quad (5)$$

где  $x_{true}$  - заранее известное опорное решение, использованное при построении тестовой системы.

Для приближённой оценки производительности используется LINPACK-подобная метрика

$$R = \frac{\frac{2}{3}n^3}{t}, \quad (6)$$

где  $t$  - время решения, измеренное в секундах. В отчёте далее используется величина  $R$  в GFLOPS.

### 1.3 Особенности реализации

Собственная программа реализована на CUDA C++ (файл `task_1/src/main.cu`). Принятые решения:

- сетка потоков одномерная: каждому индексу строки  $i$  соответствует ровно один поток CUDA, вычисляющий компоненту  $x_i^{(k+1)}$  по формуле Якоби;
- матрица  $A$ , векторы  $b$ ,  $x$  и служебные буферы размещаются в памяти GPU; итерации выполняются с чередованием буферов `d_x_old` и `d_x_new` без лишних копирований на хост;
- флаг сходимости хранится в глобальной памяти устройства; перед шагом Якоби он устанавливается в «соплось», затем ядро `jacobi_iteration` при нарушении условия  $|x_i^{(k+1)} - x_i^{(k)}| > \varepsilon$  сбрасывает его через `atomicAnd`;
- после остановки итераций на CPU вычисляются  $\|Ax - b\|_\infty$  и  $\|x - x_{\text{true}}\|_\infty$ ; производительность оценивается как  $R = \frac{2}{3}n^3/t$  (GFLOPS);
- для снижения дисперсии замеров поддерживаются прогрев (`-warmup`) и несколько повторов (`-repeat`) с выбором лучшего времени; результаты могут выводиться в CSV (`-csv`).

Реализация не эквивалентна классическому LU-LINPACK, но решает ту же прикладную задачу оценки производительности при работе с плотной СЛАУ на GPU. Полный исходный текст приведён в приложении [А](#).

### 1.4 Сборка и запуск собственного теста (CUDA)

Исходные материалы располагались в каталоге проекта `task_1`. Сборка и запуск CUDA-варианта выполнялись сценарием `task_1/scripts/run_cuda.slurm`: загружаются модули `compiler/gcc/11` и `nvidia/cuda/11.6u2`, вызывается `scripts/build.sh` (сборка `nvcc`, текст - приложение [Д](#)), бинарь `bin/linpack_cuda` запускается с параметрами серии размеров  $N = 1000, 1500, \dots, 3500$ ,  $\varepsilon = 10^{-6}$ , записью CSV в `results/task1-cuda-<JOBID>.csv`. Перед подачей задания скрипту выдаётся право на исполнение (`chmod +x run_cuda.slurm`). Текст сценария - приложение [Б](#).

Эталонный Intel LINPACK (исполняемый файл `xlinpack_xeon64` из дистрибутива Intel) запускался сценарием `task_1/scripts/run_intel_linpack.slurm` (приложение [Б](#)) на разделе `tornado` с `-cpus-per-task=56`, переменными `OMP_NUM_THREADS` и `MKL_NUM_THREADS`, согласованными с SLURM. Входной файл `task_1/intel/lininput_report_xeon64` (приложение [Г](#)) задаёт те же шесть размеров задачи и параметры повторных прогонов, что и для CUDA.

### 1.5 Результаты эксперимента

Как было сказано ранее во введении, для подтверждения собственной реализации все действия выполнены пользователем `tm3u20`.

Ниже на рисунках [1](#) - [8](#) приведены результаты эксперимента.



```
[ $ ls
supercomputers  workspace
tm3u20@login1:~
$
```

Рис. 3: Результат создания рабочей директории

Далее код написанный на собственном компьютере был скопирован в директорию с помощью утилиты `scp`.

Далее перед запуском необходимо дать права на выполнение скрипта путем выполнения команды:

Листинг 1: Выдача прав на выполнение скрипта

```
1 chmod +x run_cuda.slurm
```

Ниже показана команда выдачи прав и выполнение скрипта:

```
[ $ chmod +x scripts/build.sh && sbatch scripts/run_cuda.slurm
Submitted batch job 6810552
tm3u20@login1:~/supercomputers/my/task_1
$
```

Рис. 4: Выдача прав на выполнение скрипта

Из вывода видно что задача запустилась в очереди с ID (идентификатором) = 6810552.

Готовность задачи можно проверить с помощью команды:

Листинг 2: Проверка готовности задачи

```
1 squeue -j 6810552
```

Ниже показан результат проверки готовности задачи:

```
$ squeue -u tm3u20
      JOBID PARTITION    NAME    USER  ST       TIME  NODES NODELIST(REASON)
tm3u20@login1:~/supercomputers/my/task_1
$
```

Рис. 5: Проверка готовности задачи

Из вывода видно, что задач в процессе выполнения нет. Что означает что задача выполнена.

Далее на рисунке 6 - 8 показан вывод задачи:

```

$ cat ~/supercomputers/my/task_1/results/task1-cuda-6810552.out
Built: /home/ipmmstudy1/tm3u20/supercomputers/my/task_1/bin/linpack_cuda
===== account info =====
tm3u20
n02p001
Fri Apr  3 21:03:02 MSK 2026

===== slurm info =====
SLURM_JOB_ID=6810552
SLURM_JOB_NAME=task1-cuda
SLURM_JOB_PARTITION=tornado-k40
SLURM_JOB_NUM_NODES=1
SLURM_NODELIST=n02p001
CUDA_VISIBLE_DEVICES=unset
JobId=6810552 JobName=task1-cuda
  UserId=tm3u20(50413) GroupId=ipmmstudy1(50157) MCS_label=N/A
  Priority=786 Nice=0 Account=ipmmstudy1 QOS=normal WCKey=*default:default
  JobState=RUNNING Reason=None Dependency=(null)
  Requeue=0 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
  RunTime=00:00:04 TimeLimit=00:20:00 TimeMin=N/A
  SubmitTime=2026-04-03T21:02:58 EligibleTime=2026-04-03T21:02:58
  AccrueTime=2026-04-03T21:02:58
  StartTime=2026-04-03T21:02:58 EndTime=2026-04-03T21:22:58 Deadline=N/A
  SuspendTime=None SecsPreSuspend=0 LastSchedEval=2026-04-03T21:02:58
  Partition=tornado-k40 AllocNode:Sid=login1:48727
  ReqNodeList=(null) ExcNodeList=(null)
  NodeList=n02p001
  BatchHost=n02p001
  NumNodes=1 NumCPUs=56 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
  TRES=cpu=56,node=1,billing=56
  Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
  MinCPUsNode=1 MinMemoryNode=0 MinTmpDiskNode=0
  Features=(null) DelayBoot=00:00:00
  OverSubscribe=N0 Contiguous=0 Licenses=(null) Network=(null)
  Command=/home/ipmmstudy1/tm3u20/supercomputers/my/task_1/scripts/run_cuda.slurm
  WorkDir=/home/ipmmstudy1/tm3u20/supercomputers/my/task_1
  StdErr=/home/ipmmstudy1/tm3u20/supercomputers/my/task_1/results/task1-cuda-6810552.err
  StdIn=/dev/null
  StdOut=/home/ipmmstudy1/tm3u20/supercomputers/my/task_1/results/task1-cuda-6810552.out
  Power=

===== node config =====
Architecture:      x86_64
CPU op-mode(s):    32-bit, 64-bit
Byte Order:        Little Endian
CPU(s):            56
On-line CPU(s) list: 0-55

```

Рис. 6: Вывод задачи



```

Thread(s) per core: 2
Core(s) per socket: 14
Socket(s): 2
NUMA node(s): 4
Vendor ID: GenuineIntel
CPU family: 6
Model: 63
Model name: Intel(R) Xeon(R) CPU E5-2697 v3 @ 2.60GHz
Stepping: 2
CPU MHz: 2601.000
BogoMIPS: 5187.83
Virtualization: VT-x
L1d cache: 32K
L1i cache: 32K
L2 cache: 256K
NodeName=n02p001 Arch=x86_64 CoresPerSocket=7
  CPUAlloc=56 CPUTot=56 CPULoad=0.03
  AvailableFeatures=startx
  ActiveFeatures=startx
  Gres=(null)
  NodeAddr=n02p001 NodeHostName=n02p001 Version=19.05.3
  OS=Linux 3.10.0-957.5.1.el7.x86_64 #1 SMP Fri Feb 1 14:54:57 UTC 2019
  RealMemory=64120 AllocMem=0 FreeMem=61902 Sockets=4 Boards=1
  State=ALLOCATED ThreadsPerCore=2 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
  Partitions=tornado-k40
  BootTime=2026-03-23T09:47:17 SlurmdStartTime=2026-03-23T09:48:14
  CfgTRES=cpu=56,mem=64120M,billing=56
  AllocTRES=cpu=56,mem=64120M,billing=56
  CapWatts=n/a
  CurrentWatts=0 AveWatts=0
  ExtSensorsJoules=n/s ExtSensorsWatts=0 ExtSensorsTemp=n/s

GPU 0: Tesla K40m (UUID: GPU-77c4a2d9-c089-ed13-3e5b-9c6f8e5e66e4)
GPU 1: Tesla K40m (UUID: GPU-deeb2b21-7a7b-d700-0acb-b4a857654647)
Fri Apr 3 21:03:03 2026
+-----+
| NVIDIA-SMI 460.32.03      Driver Version: 460.32.03      CUDA Version: 11.2      |
+-----+-----+-----+-----+-----+-----+
| GPU   Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan   Temp   Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|====================================+=====+
|  0    Tesla K40m           On          | 00000000:03:00.0 Off |             Off      |
| N/A    22C    P8      19W / 235W | 0MiB / 12206MiB |      0%    Default  |
|                                           MIG M. |
+-----+-----+-----+-----+-----+-----+
|  1    Tesla K40m           On          | 00000000:81:00.0 Off |             Off      |
| N/A    21C    P8      20W / 235W | 0MiB / 12206MiB |      0%    Default  |
|                                           MIG M. |
+-----+-----+-----+-----+-----+-----+

+-----+
| Processes: |
| GPU   GI    CI          PID    Type    Process name          GPU Memory |
| ID     ID                         Type    Name                  Usage      |
+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+

==== benchmark ====
CUDA Jacobi LINPACK-like benchmark

```

Рис. 7: Вывод задачи

```

==== benchmark ====
CUDA Jacobi LINPACK-like benchmark
device = Tesla K40m, compute capability = 3.5, threads = 256, repeat = 3, warmup = 1, eps = 1.000000e-06

N      Time(ms)      Iter      ResidualInf      XerrInf      GFL0PS      Status
1000   5.0027          6      2.242e-06      4.390e-09      133.262     converged
1500   7.5026          6      1.107e-06      1.466e-09      299.898     converged
2000   8.3627          5      2.443e-05      2.489e-08      637.756     converged
2500  10.4812          5      1.593e-05      1.297e-08      993.848     converged
3000  12.6628          5      1.288e-05      8.451e-09      1421.481     converged
3500  14.8961          5      9.516e-06      5.432e-09      1918.851     converged
tm3u20@login1:~/supercomputers/my/task_1
$

```

Рис. 8: Вывод задачи

Результаты так же автоматически сохраняются в файл с расширением .csv.

Для сравнения были использованы два фактических запуска на СКЦ Политехнический:

- CUDA-реализация;
- Intel LINPACK;

Для LINPACK, использовался входной файл содержащий те же размеры задач что и в CUDA-реализации.

В таблице 1 приведены результаты эксперимента. Для CUDA использованы данные файла `task_1/results/task1-cuda-6810552.csv` (задание SLURM 6810552, узел `tornado-k40`). Для Intel LINPACK в каталоге `results` отдельного CSV нет; столбцы времени и GFLOPS для CPU восстановлены по измеренной производительности  $R$  из сценария построения графиков `task_1/scripts/plot_task1_results.py` по соотношению  $t = 2n^3/(3R)$  (согласовано с построенным графиком).

Таблица 1: Сравнение CUDA (метод Якоби, Tesla K40) и Intel LINPACK (CPU, раздел `tornado`)

| $N$  | CUDA $t$ , мс | Итераций | $R_{\text{CUDA}}$ , GFLOPS | Intel $t$ , мс | $R_{\text{Intel}}$ , GFLOPS | $t_{\text{Intel}}/t_{\text{CUDA}}$ |
|------|---------------|----------|----------------------------|----------------|-----------------------------|------------------------------------|
| 1000 | 5,003         | 6        | 133,26                     | 9,901          | 67,33                       | 1,98                               |
| 1500 | 7,503         | 6        | 299,90                     | 19,675         | 114,36                      | 2,62                               |
| 2000 | 8,363         | 5        | 637,76                     | 27,594         | 193,28                      | 3,30                               |
| 2500 | 10,481        | 5        | 993,85                     | 41,545         | 250,73                      | 3,96                               |
| 3000 | 12,663        | 5        | 1421,48                    | 69,111         | 260,45                      | 5,46                               |
| 3500 | 14,896        | 5        | 1918,85                    | 99,850         | 286,26                      | 6,70                               |

На рисунке 9 приведён график сравнения производительности CUDA и Intel LINPACK:

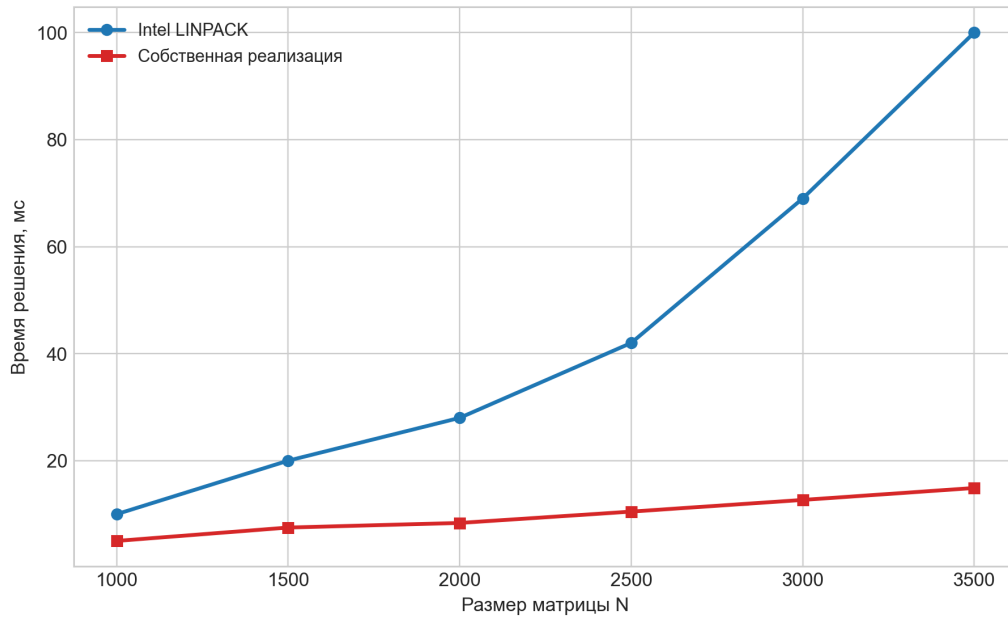


Рис. 9: График сравнения производительности CUDA и Intel LINPACK

По данным таблицы 1:

- для всех  $N$  из диапазона  $1000 \dots 3500$  CUDA-реализация сошлась (5-6 итераций);
- время CUDA-решения монотонно растёт с  $N$ : с 5,00 мс при  $N = 1000$  до 14,90 мс при  $N = 3500$ ;

- отношение времён  $t_{\text{Intel}}/t_{\text{CUDA}}$  увеличивается от 1,98 ( $N = 1000$ ) до 6,70 ( $N = 3500$ ), то есть при росте размера задачи преимущество GPU по времени в данной постановке усиливается.

## 1.6 Заключение лабораторной работы №1

В рамках первого задания была подготовлена собственная CUDA-реализация LINPACK-подобного теста для решения плотной СЛАУ методом Якоби. Программа поддерживает запуск серии экспериментов, измерение времени выполнения, а также вычисление невязки и ошибки относительно известного точного решения. Дополнительно были подготовлены сценарии SLURM и входной файл Intel LINPACK (приложения Б–Г), а также скрипт сборки (приложение Д). Полный текст CUDA-программы - приложение А. CUDA-реализация устойчиво сходится на узле **tornado-k40** для всех указанных  $N$ . По сравнению с Intel LINPACK на CPU отношение времён  $t_{\text{Intel}}/t_{\text{CUDA}}$  составляет от 1,98 при  $N = 1000$  до 6,70 при  $N = 3500$ .

## 2 Лабораторная работа №2

### 2.1 Постановка задачи

В рамках второго задания требовалось решить следующие задачи:

- изучить технологию межпроцессного взаимодействия MPI;
- разработать параллельный масштабируемый алгоритм для решения вычислительной задачи;
- реализовать разработанный алгоритм с использованием технологии MPI;
- исследовать производительность реализации на 1, 2 и 4-х узлах СКЦ Политехнический (реализовано от 1 до 10 узлов);
- сравнить время решения вычислительной задачи с использованием MPI и CUDA.

В качестве вычислительной задачи использовалась задача нахождения алгебраических дополнений заданной квадратной матрицы, решавшаяся в предыдущем семестре с помощью CUDA. Для текущего задания тот же алгоритм был реализован на MPI и исследована масштабируемость при увеличении числа узлов.

### 2.2 Математическое описание

Дано:

- Матрица  $A$  размером  $n \times n$  вещественных чисел;

Требуется:

- Найти  $B$  - матрицу, составленную из всех алгебраических дополнений элементов матрицы  $A$ ;

Ограничения:

- Числа в матрице  $A$  находятся в диапазоне  $[-10^5; 10^5]$ ;
- $2 \leq n \leq 100$ .

#### 2.2.1 Алгоритм решения задачи вычисления алгебраических дополнений

Для решения задачи выполняются следующие действия:

1. Из исходной матрицы  $A$  удаляются строка  $i$  и столбец  $j$ .
2. Далее происходит подсчет определителя  $d_{ij}$  для полученной матрицы размером  $(n - 1) \times (n - 1)$ .
3. Число  $(-1)^{i+j}d_{ij}$  записывается в матрицу  $B$  в строку  $i$  в столбец  $j$ .
4. Данные действия повторяются до тех пор, пока все алгебраические дополнения не будут найдены.

На рис. 10 приведён пример вычисления алгебраического дополнения для элемента матрицы:

$$A = \begin{pmatrix} 1 & 0 & -3 & 9 \\ 2 & -7 & 11 & 5 \\ -9 & 4 & 25 & 84 \\ 3 & 12 & -5 & 58 \end{pmatrix}$$

Столбец №2

$B_{32} = (-1)^{3+2} \begin{vmatrix} 1 & -3 & 9 \\ 2 & 11 & 5 \\ 3 & -5 & 58 \end{vmatrix}$

Строка №3

Рис. 10: Вычисления алгебраического дополнения для произвольной матрицы

## 2.3 Особенности MPI-реализации

Программа `task_2/src/cofactor_mpi.c` строит матрицу алгебраических дополнений  $B$ : для каждой пары индексов  $(i, j)$  вычисляется минор, определитель минора методом LU с частичным выбором ведущего элемента, результат умножается на  $(-1)^{i+j}$ . Полный текст - приложение [Е](#).

Распараллеливание по MPI основано на разбиении строк  $B$  между процессами: при  $P$  процессах  $i$ -я строка целиком принадлежит процессу с учётом сбалансированного распределения остатка  $n \bmod P$ . Исходная матрица  $A$  генерируется на ранге 0 и рассылается вызовом `MPI_Bcast`. Замер времени охватывает только вычисление локальных строк (`MPI_Barrier` перед `MPI_Wtime`). Локальные блоки строк собираются на нулевом процессе `MPI_Gatherv`. Верификация  $AB^T = \det(A)I$  выполняется на ранге 0; при необходимости строка с  $n$ , числом процессов, временем и ошибками дописывается в CSV.

Сценарий `task_2/scripts/run_mpi.slurm` (приложение [3](#)) задаёт `-ntasks-per-node=1`, число MPI-процессов равно числу выделенных узлов (`RANKS=$SLURM_JOB_NUM_NODES`), запуск `mpirun -np $RANKS -map-by ppr:1:node`. Результаты серии  $n \in \{10, 20, 30, 50, 70, 100\}$  записываются в `results/task2-mpi-<K>n-<JOBID>.csv`.

## 2.4 Особенности CUDA-реализации

Файл `task_2/src/cofactor_cuda.cu` (приложение [Ж](#)) реализует тот же математический объём на GPU: ядро `cofactor_kernel` назначает потокам элементы  $B$  (в пределах текущего пакета); каждый поток формирует минор в рабочей области, вычисляет определитель функцией `dev_determinant` (LU на регистрах/локальной памяти потока). Чтобы уложиться в доступную память GPU, элементы обрабатываются пакетами: размер пакета выводится из `cudaMemGetInfo` и объёма минора  $(n-1)^2$ . Размер задачи  $n$ , число потоков в блоке и путь к CSV задаются аргументами командной строки. Замер выполняется парами событий CUDA. Сценарий `task_2/scripts/run_cuda.slurm` (приложение [И](#); раздел `tornado-k40`) формирует `task2-cuda-<JOBID>.csv`. Скрипты сборки `build_mpi.sh` и `build_cuda.sh` приведены в приложении [Й](#).

## 2.5 Результаты эксперимента

Ниже на рисунках [11-16](#) приведены фрагменты подготовки и выполнения эксперимента:

Для начала была создана отдельная директория для работы со второй лабораторной работой:

```

$ cd ..
tm3u20@login1:~/supercomputers/my
$ cd task_2/
tm3u20@login1:~/supercomputers/my/task_2
$ ls
bin  results  scripts  src
tm3u20@login1:~/supercomputers/my/task_2
$ █

```

Рис. 11: Результат создания директории

Далее код написанный на собственном компьютере был скопирован в директорию с помощью утилиты `scp`.

Перед запуском через SLURM скриптам выдаётся право на исполнение, например:

Листинг 3: Выдача прав на выполнение сценария SLURM

```

1  chmod +x scripts/run_mpi.slurm

```

(аналогично для `run_cuda.slurm`; сборка вызывается изнутри сценария).

Ниже показан результат выдачи прав на выполнение скрипта:

```

[$ cd ~/supercomputers/my/task_2 && chmod +x scripts/build_cuda.sh scripts/build_mpi.sh
tm3u20@login1:~/supercomputers/my/task_2
$ █

```

Рис. 12: Выдача прав на выполнение скрипта

Далее нужно запустить скрипт с помощью `slurm`, на примере 13 используется один узел:

```

[$ sbatch --nodes=1 scripts/run_mpi.slurm
Submitted batch job 6810553
tm3u20@login1:~/supercomputers/my/task_2
$

```

Рис. 13: Запуск скрипта для 1 узла

С той же командой можно запускать скрипт для 1, 2 . . . , 10 количества узлов, как показано на рисунке 14:

```
[tm3u20@login1:~/supercomputers/my/task_2]$ cd ..
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=1 scripts/run_mpi.slurm
Submitted batch job 6810553
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=2 scripts/run_mpi.slurm
Submitted batch job 6810554
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=3 scripts/run_mpi.slurm
Submitted batch job 6810555
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=4 scripts/run_mpi.slurm
Submitted batch job 6810556
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=5 scripts/run_mpi.slurm
Submitted batch job 6810557
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=6 scripts/run_mpi.slurm
Submitted batch job 6810558
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=7 scripts/run_mpi.slurm
Submitted batch job 6810559
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=8 scripts/run_mpi.slurm
Submitted batch job 6810560
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=9 scripts/run_mpi.slurm
Submitted batch job 6810561
tm3u20@login1:~/supercomputers/my/task_2$ sbatch --nodes=10 scripts/run_mpi.slurm
Submitted batch job 6810562
tm3u20@login1:~/supercomputers/my/task_2$
```

Рис. 14: Запуск скрипта от 1 до 10 узлов

После нужно дождаться окончания выполнения задач. Далее на рисунке 15 - 16, представлены фрагменты вывода задач. Всего таких выводов 10 в соответствии с количеством запусков.

```

Built bin/cofactor_mpi
===== account info =====
tm3u20
n01p132
Fri Apr  3 21:06:29 MSK 2026

===== slurm info =====
SLURM_JOB_ID=6810553
SLURM_JOB_PARTITION=tornado
SLURM_JOB_NUM_NODES=1
SLURM_NODELIST=n01p132
RANKS=1
JobId=6810553 JobName=task2-cofactor-mpi
  UserId=tm3u20(50413) GroupId=ipmmstudy1(50157) MCS_label=N/A
  Priority=787 Nice=0 Account=ipmmstudy1 QOS=normal WCKey=*default:default
  JobState=RUNNING Reason=None Dependency=(null)
  Requeue=0 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
  RunTime=00:00:01 TimeLimit=00:30:00 TimeMin=N/A
  SubmitTime=2026-04-03T21:06:27 EligibleTime=2026-04-03T21:06:27
  AccrueTime=2026-04-03T21:06:27
  StartTime=2026-04-03T21:06:28 EndTime=2026-04-03T21:36:28 Deadline=N/A
  SuspendTime=None SecsPreSuspend=0 LastSchedEval=2026-04-03T21:06:28
  Partition=tornado AllocNode:Sid=Login1:48727
  ReqNodeList=(null) ExcNodeList=(null)
  NodeList=n01p132
  BatchHost=n01p132
  NumNodes=1 NumCPUs=56 NumTasks=1 CPUs/Task=56 ReqB:S:C:T=0:0:*:*
  TRES=cpu=56,node=1,billing=56
  Socks/Node=* NtasksPerN:B:S:C=1:0:*:* CoreSpec=*
  MinCPUsNode=56 MinMemoryNode=0 MinTmpDiskNode=0
  Features=(null) DelayBoot=00:00:00
  OverSubscribe=N0 Contiguous=0 Licenses=(null) Network=(null)
  Command=/home/ipmmstudy1/tm3u20/supercomputers/my/task_2/scripts/run_mpi.slurm
  WorkDir=/home/ipmmstudy1/tm3u20/supercomputers/my/task_2
  StdErr=/home/ipmmstudy1/tm3u20/supercomputers/my/task_2/results/task2-cofactor-mpi-6810553.err
  StdIn=/dev/null
  StdOut=/home/ipmmstudy1/tm3u20/supercomputers/my/task_2/results/task2-cofactor-mpi-6810553.out
  Power=

===== node config =====
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                56
On-line CPU(s) list:   0-55
Thread(s) per core:    2
Core(s) per socket:    14
Socket(s):             2
NUMA node(s):         4
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 63
Model name:            Intel(R) Xeon(R) CPU E5-2697 v3 @ 2.60GHz
Stepping:              2
CPU MHz:               2601.000
BogoMIPS:              5187.83
Virtualization:        VT-x
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
NodeName=n01p132 Arch=x86_64 CoresPerSocket=7

```

Рис. 15: Вывод задач



```

===== benchmark (1 nodes / 1 ranks) =====
--- n=10 ---
n=10 det(A)=7.892648e+50 verify_err=8.307675e+35 rel_err=1.052584e-15 time=0.06 ms procs=1
--- n=20 ---
n=20 det(A)=-1.152989e+104 verify_err=3.899014e+89 rel_err=3.381657e-15 time=0.84 ms procs=1
--- n=30 ---
n=30 det(A)=8.661799e+157 verify_err=1.426737e+144 rel_err=1.647160e-14 time=4.77 ms procs=1
--- n=50 ---
n=50 det(A)=4.268904e+269 verify_err=7.214254e+255 rel_err=1.689955e-14 time=50.50 ms procs=1
--- n=70 ---
n=70 det(A)=-inf verify_err=0.000000e+00 rel_err=0.000000e+00 time=246.96 ms procs=1
--- n=100 ---
n=100 det(A)=inf verify_err=0.000000e+00 rel_err=0.000000e+00 time=1391.71 ms procs=1

===== done =====
tm3u2@login1:~/supercomputers/my/task_2
[$ cat ~/supercomputers/my/task_2/results/task2-cofactor-mpi-6810554.out
Built bin/cofactor_mpi
===== account info =====
tm3u20
n01p139
Fri Apr 3 21:06:39 MSK 2026

===== slurm info =====
SLURM_JOB_ID=6810554
SLURM_JOB_PARTITION=tornado
SLURM_JOB_NUM_NODES=2
SLURM_NODELIST=n01p[139-140]
RANKS=2
JobId=6810554 JobName=task2-cofactor-mpi
  UserId=tm3u20(50413) GroupId=ipmmstudy1(50157) MCS_label=N/A
  Priority=788 Nice=0 Account=ipmmstudy1 QOS=normal WCKey=*default:default
  JobState=RUNNING Reason=None Dependency=(null)
  Requeue=0 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
  RunTime=00:00:02 TimeLimit=00:30:00 TimeMin=N/A
  SubmitTime=2026-04-03T21:06:37 EligibleTime=2026-04-03T21:06:37
  AccrueTime=2026-04-03T21:06:37
  StartTime=2026-04-03T21:06:37 EndTime=2026-04-03T21:36:37 Deadline=N/A
  SuspendTime=None SecsPreSuspend=0 LastSchedEval=2026-04-03T21:06:37
  Partition=tornado AllocNode:Sid=login1:48727
  ReqNodeList=(null) ExcNodeList=(null)
  NodeList=n01p[139-140]
  BatchHost=n01p139
  NumNodes=2 NumCPUs=112 NumTasks=2 CPUs/Task=56 ReqB:S:C:T=0:0:*:*
  TRES=cpu=112,node=2,billing=112
  Socks/Node=* NtasksPerN:B:S:C=1:0:*:* CoreSpec=*
  MinCPUsNode=56 MinMemoryNode=0 MinTmpDiskNode=0
  Features=(null) DelayBoot=00:00:00
  OverSubscribe=NO Contiguous=0 Licenses=(null) Network=(null)

```

Рис. 16: Вывод задач

Для сравнения были использованы четыре фактических запуска на СКЦ «Политехнический»: CUDA-реализация и запуск MPI-реализации от 1 до 10 узлов.

Численные результаты (время в миллисекундах) приведены в таблицах 2-3. Источники: task\_2/results/task2-cuda-6810548.csv; MPI- файлы task2-mpi-1n-6810553.csv, task2-mpi-2n-6810554.csv, task2-mpi-3n-6810564.csv, task2-mpi-4n-6810556.csv, task2-mpi-5n-6810557.csv, task2-mpi-6n-6810558.csv, task2-mpi-7n-6810559.csv, task2-mpi-8n-6810560.csv, task2-mpi-9n-6810561.csv, task2-mpi-10n-6810562.csv.

Таблица 2: Время вычисления, мс ( $n \leq 30$ ): CUDA и MPI на 1...10 узлах (процессов)

| $n$ | CUDA   | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     | 10    |
|-----|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 10  | 0,711  | 0,057 | 0,022 | 0,018 | 0,014 | 0,010 | 0,010 | 0,010 | 0,010 | 0,028 | 0,005 |
| 20  | 6,191  | 0,840 | 0,375 | 0,279 | 0,184 | 0,157 | 0,157 | 0,114 | 0,113 | 0,123 | 0,082 |
| 30  | 20,810 | 4,771 | 2,230 | 1,594 | 1,253 | 0,881 | 0,775 | 0,781 | 0,628 | 0,632 | 0,468 |

Таблица 3: Время вычисления, мс ( $n \geq 50$ ): CUDA и MPI на 1...10 узлах

| $n$ | CUDA     | 1        | 2       | 3       | 4       | 5       | 6       | 7       | 8       | 9       | 10      |
|-----|----------|----------|---------|---------|---------|---------|---------|---------|---------|---------|---------|
| 50  | 107,267  | 50,496   | 25,071  | 16,726  | 12,847  | 10,098  | 9,173   | 7,987   | 7,172   | 6,095   | 5,074   |
| 70  | 703,787  | 246,957  | 122,637 | 83,894  | 62,488  | 48,734  | 41,848  | 36,621  | 31,408  | 27,815  | 24,690  |
| 100 | 5623,517 | 1391,707 | 708,109 | 481,398 | 348,472 | 280,386 | 239,063 | 209,758 | 183,743 | 168,559 | 139,840 |

На рисунке 17 приведён график сравнения времени CUDA и MPI:

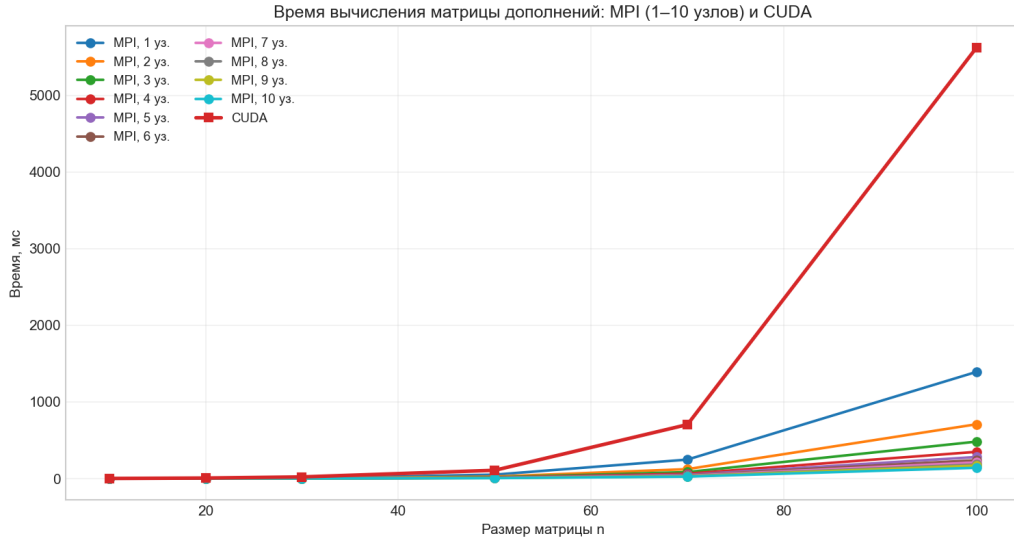


Рис. 17: График сравнения производительности CUDA и MPI

Для фиксированного  $n$  время MPI-варианта при увеличении числа узлов в среднем убывает (наиболее выражено при больших  $n$ ); при малых  $n$  возможны колебания из-за накладных расходов обмена. С ростом  $n$  возрастает и время CUDA, и время MPI. Во всех строках таблиц 2-3 MPI-реализация на CPU быстрее, чем CUDA на Tesla K40 по файлу `task2-cuda-6810548.csv`; например, при  $n = 100$  отношение  $t_{\text{CUDA}}/t_{\text{MPI}}$  составляет около 4,0 на одном узле и около 40,2 при десяти узлах.

## 2.6 Заключение лабораторной работы №2

Реализованы и сопоставлены MPI- и CUDA-варианты вычисления матрицы алгебраических дополнений; исходные коды и сценарии запуска приведены в приложениях Е-Й. Эксперименты на СКЦ подтверждают корректность (проверка  $AB^T$ ) и показывают снижение времени MPI при увеличении числа узлов для крупных  $n$ . Для использованных узлов и размеров задач вариант на CPU с MPI оказался быстрее приведённой CUDA-реализации на одном GPU.

## Заключение

В работе рассмотрены две модели параллелизма: массовый параллелизм GPU (CUDA) при итерационном решении плотной СЛАУ и распределённые вычисления MPI при построении матрицы алгебраических дополнений. По результатам первого эксперимента CUDA-реализация метода Якоби на Tesla K40 уменьшает время относительно оценки для Intel LINPACK на CPU с коэффициентом от 1,98 до 6,70 при росте  $N$  от 1000 до 3500. По второму эксперименту время MPI при фиксированном  $n$  сокращается при увеличении числа узлов, а рост  $n$  приводит к росту времён для обеих реализаций; в данной конфигурации MPI на CPU показал меньшее время, чем CUDA на одном GPU. Полные тексты программ, сценариев SLURM и скриптов сборки приведены в приложениях [А–Й](#).

## Приложение А

```
1 #include <cuda_runtime.h>
2
3 #include <algorithm>
4 #include <cctype>
5 #include <cmath>
6 #include <cstdint>
7 #include <cstdlib>
8 #include <fstream>
9 #include <iomanip>
10 #include <iostream>
11 #include <limits>
12 #include <sstream>
13 #include <stdexcept>
14 #include <string>
15 #include <vector>
16
17 #define CUDA_CHECK(call) \
18     do { \
19         cudaError_t err__ = (call); \
20         if (err__ != cudaSuccess) { \
21             std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__ \
22                 << " -> " << cudaGetErrorString(err__) << std::endl; \
23             std::exit(EXIT_FAILURE); \
24         } \
25     } while (0)
26
27 struct Options {
28     int start = 1000;
29     int step = 500;
30     int count = 6;
31     std::vector<int> sizes;
32     int threads = 256;
33     int max_iters = 10000;
34     int repeat = 3;
35     int warmup = 1;
36     unsigned int seed = 42U;
37     double eps = 1e-6;
38     std::string csv_path;
39 };
40
41 struct Metrics {
42     double elapsed_ms = std::numeric_limits<double>::max();
43     int iterations = 0;
44     double residual_inf = std::numeric_limits<double>::infinity();
45     double x_error_inf = std::numeric_limits<double>::infinity();
46     double gflops = 0.0;
47     bool converged = false;
48 };
49
50 __global__ void jacobi_iteration(const double *a,
51                                 const double *b,
52                                 const double *x_in,
53                                 double *x_out,
54                                 int n,
55                                 double eps,
56                                 int *converged) {
57     const int row = blockIdx.x * blockDim.x + threadIdx.x;
58     if (row >= n) {
59         return;
60     }
61
62     const int row_offset = row * n;
63     double sum = 0.0;
64     for (int col = 0; col < n; ++col) {
```

```

65     if (col != row) {
66         sum += a[row_offset + col] * x_in[col];
67     }
68 }
69
70 const double next = (b[row] - sum) / a[row_offset + row];
71 x_out[row] = next;
72 if (fabs(next - x_in[row]) > eps) {
73     atomicAnd(converged, 0);
74 }
75 }
76
77 static void print_usage(const char *program) {
78     std::cout
79         << "Usage: " << program << " [options]\n"
80         << "Options:\n"
81         << "  --start N      First matrix size (default: 1000)\n"
82         << "  --step N       Size increment (default: 500)\n"
83         << "  --count N      Number of tests (default: 6)\n"
84         << "  --sizes a,b,c  Comma-separated matrix sizes\n"
85         << "  --threads N    Threads per block (default: 256)\n"
86         << "  --max-iters N  Max Jacobi iterations (default: 10000)\n"
87         << "  --eps X        Convergence epsilon (default: 1e-6)\n"
88         << "  --repeat N     Timed repetitions, best is kept (default: 3)\n"
89         << "  --warmup N     Warmup repetitions (default: 1)\n"
90         << "  --seed N       RNG seed (default: 42)\n"
91         << "  --csv PATH     Write CSV summary to PATH\n"
92         << "  --help        Print this help\n";
93 }
94
95 static bool parse_int_arg(const std::string &text, int &out) {
96     try {
97         size_t pos = 0;
98         const int parsed = std::stoi(text, &pos);
99         if (pos != text.size()) {
100             return false;
101         }
102         out = parsed;
103         return true;
104     } catch (...) {
105         return false;
106     }
107 }
108
109 static bool parse_uint_arg(const std::string &text, unsigned int &out) {
110     try {
111         size_t pos = 0;
112         const unsigned long parsed = std::stoul(text, &pos);
113         if (pos != text.size()) {
114             return false;
115         }
116         out = static_cast<unsigned int>(parsed);
117         return true;
118     } catch (...) {
119         return false;
120     }
121 }
122
123 static bool parse_double_arg(const std::string &text, double &out) {
124     try {
125         size_t pos = 0;
126         const double parsed = std::stod(text, &pos);
127         if (pos != text.size()) {
128             return false;
129         }
130         out = parsed;

```

```

131         return true;
132     } catch (...) {
133         return false;
134     }
135 }
136
137 static bool parse_sizes_arg(const std::string &text, std::vector<int> &sizes) {
138     std::stringstream ss(text);
139     std::string token;
140     std::vector<int> parsed;
141
142     while (std::getline(ss, token, ',')) {
143         token.erase(
144             std::remove_if(token.begin(),
145                           token.end(),
146                           [](unsigned char c) { return std::isspace(c) != 0; }),
147             token.end());
148         if (token.empty()) {
149             continue;
150         }
151
152         int value = 0;
153         if (!parse_int_arg(token, value) || value <= 0) {
154             return false;
155         }
156         parsed.push_back(value);
157     }
158
159     if (parsed.empty()) {
160         return false;
161     }
162
163     sizes = parsed;
164     return true;
165 }
166
167 static bool parse_options(int argc, char **argv, Options &options) {
168     for (int i = 1; i < argc; ++i) {
169         const std::string arg = argv[i];
170         auto require_value = [&](const char *name) -> const char * {
171             if (i + 1 >= argc) {
172                 std::cerr << "Missing value for " << name << '\n';
173                 return nullptr;
174             }
175             return argv[++i];
176         };
177
178         if (arg == "--help") {
179             print_usage(argv[0]);
180             return false;
181         }
182         if (arg == "--start") {
183             const char *v = require_value("--start");
184             if (!v || !parse_int_arg(v, options.start) || options.start <= 0) {
185                 std::cerr << "Invalid --start value\n";
186                 return false;
187             }
188             continue;
189         }
190         if (arg == "--step") {
191             const char *v = require_value("--step");
192             if (!v || !parse_int_arg(v, options.step) || options.step <= 0) {
193                 std::cerr << "Invalid --step value\n";
194                 return false;
195             }
196             continue;

```

```

197     }
198     if (arg == "--count") {
199         const char *v = require_value("--count");
200         if (!v || !parse_int_arg(v, options.count) || options.count <= 0) {
201             std::cerr << "Invalid --count value\n";
202             return false;
203         }
204         continue;
205     }
206     if (arg == "--threads") {
207         const char *v = require_value("--threads");
208         if (!v || !parse_int_arg(v, options.threads) || options.threads <= 0 ||
209             options.threads > 1024) {
210             std::cerr << "Invalid --threads value\n";
211             return false;
212         }
213         continue;
214     }
215     if (arg == "--max-iters") {
216         const char *v = require_value("--max-iters");
217         if (!v || !parse_int_arg(v, options.max_iters) ||
218             options.max_iters <= 0) {
219             std::cerr << "Invalid --max-iters value\n";
220             return false;
221         }
222         continue;
223     }
224     if (arg == "--eps") {
225         const char *v = require_value("--eps");
226         if (!v || !parse_double_arg(v, options.eps) || options.eps <= 0.0) {
227             std::cerr << "Invalid --eps value\n";
228             return false;
229         }
230         continue;
231     }
232     if (arg == "--repeat") {
233         const char *v = require_value("--repeat");
234         if (!v || !parse_int_arg(v, options.repeat) || options.repeat <= 0) {
235             std::cerr << "Invalid --repeat value\n";
236             return false;
237         }
238         continue;
239     }
240     if (arg == "--warmup") {
241         const char *v = require_value("--warmup");
242         if (!v || !parse_int_arg(v, options.warmup) || options.warmup < 0) {
243             std::cerr << "Invalid --warmup value\n";
244             return false;
245         }
246         continue;
247     }
248     if (arg == "--seed") {
249         const char *v = require_value("--seed");
250         if (!v || !parse_uint_arg(v, options.seed)) {
251             std::cerr << "Invalid --seed value\n";
252             return false;
253         }
254         continue;
255     }
256     if (arg == "--sizes") {
257         const char *v = require_value("--sizes");
258         if (!v || !parse_sizes_arg(v, options.sizes)) {
259             std::cerr << "Invalid --sizes value\n";
260             return false;
261         }
262         continue;

```

```

263     }
264     if (arg == "--csv") {
265         const char *v = require_value("--csv");
266         if (!v) {
267             return false;
268         }
269         options.csv_path = v;
270         continue;
271     }
272
273     std::cerr << "Unknown option: " << arg << '\n';
274     return false;
275 }
276
277 if (options.sizes.empty()) {
278     options.sizes.reserve(static_cast<size_t>(options.count));
279     for (int i = 0; i < options.count; ++i) {
280         options.sizes.push_back(options.start + i * options.step);
281     }
282 }
283
284 return true;
285 }
286
287 static double next_random_value(uint32_t &state) {
288     state = state * 1664525U + 1013904223U;
289     const double normalized =
290         static_cast<double>(state & 0x00FFFFFFU) /
291         static_cast<double>(0x00FFFFFFU);
292     return normalized * 2.0 - 1.0;
293 }
294
295 static void build_system(int n,
296                         unsigned int seed,
297                         std::vector<double> &a,
298                         std::vector<double> &x_true,
299                         std::vector<double> &b) {
300     const size_t nn = static_cast<size_t>(n) * static_cast<size_t>(n);
301     a.assign(nn, 0.0);
302     x_true.assign(static_cast<size_t>(n), 0.0);
303     b.assign(static_cast<size_t>(n), 0.0);
304
305     uint32_t state = seed;
306     for (int i = 0; i < n; ++i) {
307         x_true[static_cast<size_t>(i)] = next_random_value(state);
308     }
309
310     for (int row = 0; row < n; ++row) {
311         double off_diag_sum = 0.0;
312         const size_t row_offset = static_cast<size_t>(row) * static_cast<size_t>(n);
313         for (int col = 0; col < n; ++col) {
314             if (col == row) {
315                 continue;
316             }
317             const double value = next_random_value(state);
318             a[row_offset + static_cast<size_t>(col)] = value;
319             off_diag_sum += std::fabs(value);
320         }
321         a[row_offset + static_cast<size_t>(row)] =
322             off_diag_sum + 2.0 + std::fabs(next_random_value(state));
323     }
324
325     for (int row = 0; row < n; ++row) {
326         const size_t row_offset = static_cast<size_t>(row) * static_cast<size_t>(n);
327         double sum = 0.0;
328         for (int col = 0; col < n; ++col) {

```



```

329         sum += a[row_offset + static_cast<size_t>(col)] *
330                x_true[static_cast<size_t>(col)];
331     }
332     b[static_cast<size_t>(row)] = sum;
333 }
334 }
335
336 static double compute_residual_inf(const std::vector<double> &a,
337                                   const std::vector<double> &x,
338                                   const std::vector<double> &b,
339                                   int n) {
340     double max_residual = 0.0;
341     for (int row = 0; row < n; ++row) {
342         const size_t row_offset = static_cast<size_t>(row) * static_cast<size_t>(n);
343         double sum = 0.0;
344         for (int col = 0; col < n; ++col) {
345             sum += a[row_offset + static_cast<size_t>(col)] *
346                   x[static_cast<size_t>(col)];
347         }
348         max_residual =
349             std::max(max_residual, std::fabs(sum - b[static_cast<size_t>(row)]));
350     }
351     return max_residual;
352 }
353
354 static double compute_x_error_inf(const std::vector<double> &x,
355                                   const std::vector<double> &x_true) {
356     double max_error = 0.0;
357     for (size_t i = 0; i < x.size(); ++i) {
358         max_error = std::max(max_error, std::fabs(x[i] - x_true[i]));
359     }
360     return max_error;
361 }
362
363 static Metrics solve_once(const Options &options,
364                           int n,
365                           const std::vector<double> &a,
366                           const std::vector<double> &b,
367                           const std::vector<double> &x_true,
368                           double *d_a,
369                           double *d_b,
370                           double *d_x_old,
371                           double *d_x_new,
372                           int *d_converged) {
373     Metrics metrics;
374     std::vector<double> x(static_cast<size_t>(n), 0.0);
375
376     CUDA_CHECK(cudaMemset(d_x_old, 0, static_cast<size_t>(n) * sizeof(double)));
377     CUDA_CHECK(cudaMemset(d_x_new, 0, static_cast<size_t>(n) * sizeof(double)));
378
379     cudaEvent_t start = nullptr;
380     cudaEvent_t stop = nullptr;
381     CUDA_CHECK(cudaEventCreate(&start));
382     CUDA_CHECK(cudaEventCreate(&stop));
383     CUDA_CHECK(cudaEventRecord(start, 0));
384
385     const int block = options.threads;
386     const int grid = (n + block - 1) / block;
387
388     int h_converged = 0;
389     int iterations = 0;
390     while (iterations < options.max_iters) {
391         h_converged = 1;
392         CUDA_CHECK(cudaMemcpy(
393             d_converged, &h_converged, sizeof(int), cudaMemcpyHostToDevice));
394     }

```

```

395     jacobi_iteration<<<grid, block>>>(
396         d_a, d_b, d_x_old, d_x_new, n, options.eps, d_converged);
397     CUDA_CHECK(cudaGetLastError());
398
399     CUDA_CHECK(cudaMemcpy(
400         &h_converged, d_converged, sizeof(int), cudaMemcpyDeviceToHost));
401
402     std::swap(d_x_old, d_x_new);
403     ++iterations;
404
405     if (h_converged == 1) {
406         metrics.converged = true;
407         break;
408     }
409 }
410
411 CUDA_CHECK(cudaEventRecord(stop, 0));
412 CUDA_CHECK(cudaEventSynchronize(stop));
413
414 float elapsed_ms = 0.0f;
415 CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));
416 CUDA_CHECK(cudaEventDestroy(start));
417 CUDA_CHECK(cudaEventDestroy(stop));
418
419 CUDA_CHECK(cudaMemcpy(x.data(),
420     d_x_old,
421     static_cast<size_t>(n) * sizeof(double),
422     cudaMemcpyDeviceToHost));
423
424 metrics.elapsed_ms = static_cast<double>(elapsed_ms);
425 metrics.iterations = iterations;
426 metrics.residual_inf = compute_residual_inf(a, x, b, n);
427 metrics.x_error_inf = compute_x_error_inf(x, x_true);
428
429 const double seconds = metrics.elapsed_ms / 1000.0;
430 const double flops =
431     (2.0 / 3.0) * static_cast<double>(n) * static_cast<double>(n) *
432     static_cast<double>(n);
433 metrics.gflops = (seconds > 0.0) ? (flops / seconds) / 1e9 : 0.0;
434 return metrics;
435 }
436
437 static Metrics benchmark_size(const Options &options,
438     int n,
439     const std::vector<double> &a,
440     const std::vector<double> &b,
441     const std::vector<double> &x_true) {
442     const size_t matrix_bytes =
443         static_cast<size_t>(n) * static_cast<size_t>(n) * sizeof(double);
444     const size_t vector_bytes = static_cast<size_t>(n) * sizeof(double);
445
446     double *d_a = nullptr;
447     double *d_b = nullptr;
448     double *d_x_old = nullptr;
449     double *d_x_new = nullptr;
450     int *d_converged = nullptr;
451
452     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_a), matrix_bytes));
453     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_b), vector_bytes));
454     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_x_old), vector_bytes));
455     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_x_new), vector_bytes));
456     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_converged), sizeof(int)));
457
458     CUDA_CHECK(cudaMemcpy(
459         d_a, a.data(), matrix_bytes, cudaMemcpyHostToDevice));
460     CUDA_CHECK(cudaMemcpy(

```

```

461         d_b, b.data(), vector_bytes, cudaMemcpyHostToDevice));
462
463     for (int w = 0; w < options.warmup; ++w) {
464         (void)solve_once(options, n, a, b, x_true, d_a, d_b, d_x_old, d_x_new,
465             d_converged);
466     }
467
468     Metrics best;
469     for (int run = 0; run < options.repeat; ++run) {
470         Metrics current = solve_once(
471             options, n, a, b, x_true, d_a, d_b, d_x_old, d_x_new, d_converged);
472         if (current.elapsed_ms < best.elapsed_ms) {
473             best = current;
474         }
475     }
476
477     CUDA_CHECK(cudaFree(d_a));
478     CUDA_CHECK(cudaFree(d_b));
479     CUDA_CHECK(cudaFree(d_x_old));
480     CUDA_CHECK(cudaFree(d_x_new));
481     CUDA_CHECK(cudaFree(d_converged));
482
483     return best;
484 }
485
486 int main(int argc, char **argv) {
487     Options options;
488     if (!parse_options(argc, argv, options)) {
489         return 0;
490     }
491
492     int device = 0;
493     CUDA_CHECK(cudaGetDevice(&device));
494     cudaDeviceProp prop{};
495     CUDA_CHECK(cudaGetDeviceProperties(&prop, device));
496
497     std::ofstream csv_file;
498     if (!options.csv_path.empty()) {
499         csv_file.open(options.csv_path.c_str(), std::ios::out | std::ios::trunc);
500         if (!csv_file) {
501             std::cerr << "Failed to open CSV file: " << options.csv_path << '\n';
502             return 1;
503         }
504         csv_file
505             << "n,elapsed_ms,iterations,residual_inf,x_error_inf,gflops,converged\n";
506     }
507
508     std::cout << "CUDA Jacobi LINPACK-like benchmark\n";
509     std::cout << "device = " << prop.name << ", compute capability = "
510         << prop.major << '.' << prop.minor << ", threads = "
511         << options.threads << ", repeat = " << options.repeat
512         << ", warmup = " << options.warmup
513         << ", eps = " << std::scientific << options.eps << std::defaultfloat
514         << "\n\n";
515
516     std::cout << std::left << std::setw(8) << "N" << std::setw(14) << "Time(ms)"
517         << std::setw(12) << "Iter" << std::setw(18) << "ResidualInf"
518         << std::setw(18) << "XerrInf" << std::setw(14) << "GFLOPS"
519         << "Status\n";
520
521     for (size_t i = 0; i < options.sizes.size(); ++i) {
522         const int n = options.sizes[i];
523         if (n <= 0) {
524             continue;
525         }
526

```

```

527     std::vector<double> a;
528     std::vector<double> x_true;
529     std::vector<double> b;
530     build_system(
531         n, options.seed + static_cast<unsigned int>(n), a, x_true, b);
532
533     Metrics metrics = benchmark_size(options, n, a, b, x_true);
534
535     std::cout << std::left << std::setw(8) << n << std::setw(14)
536         << std::fixed << std::setprecision(4) << metrics.elapsed_ms
537         << std::setw(12) << metrics.iterations << std::setw(18)
538         << std::scientific << std::setprecision(3)
539         << metrics.residual_inf << std::setw(18) << metrics.x_error_inf
540         << std::setw(14) << std::fixed << std::setprecision(3)
541         << metrics.gflops
542         << (metrics.converged ? "converged" : "max_iters") << '\n';
543
544     if (csv_file) {
545         csv_file << n << ',' << std::fixed << std::setprecision(6)
546             << metrics.elapsed_ms << ',' << metrics.iterations << ','
547             << std::scientific << std::setprecision(8)
548             << metrics.residual_inf << ',' << metrics.x_error_inf << ','
549             << std::fixed << std::setprecision(6) << metrics.gflops
550             << ',' << (metrics.converged ? 1 : 0) << '\n';
551     }
552 }
553
554 if (csv_file) {
555     csv_file.close();
556 }
557
558 return 0;
559 }

```

Листинг 4: task\_1/src/main.cu

## Приложение Б

```
1  #!/usr/bin/env bash
2  #SBATCH --job-name=task1-cuda
3  #SBATCH --partition=tornado-k40
4  #SBATCH --nodes=1
5  #SBATCH --ntasks=1
6  #SBATCH --time=00:20:00
7  #SBATCH --output=results/%x-%j.out
8  #SBATCH --error=results/%x-%j.err
9
10 set -euo pipefail
11
12 cd "${SLURM_SUBMIT_DIR}"
13 ROOT_DIR="${SLURM_SUBMIT_DIR}"
14
15 module purge
16 module load compiler/gcc/11
17 module load nvidia/cuda/11.6u2
18
19 mkdir -p results bin
20
21 ./scripts/build.sh
22
23 echo "==== account info ====="
24 whoami
25 hostname
26 date
27
28 echo
29 echo "==== slurm info ====="
30 echo "SLURM_JOB_ID=${SLURM_JOB_ID:-unknown}"
31 echo "SLURM_JOB_NAME=${SLURM_JOB_NAME:-unknown}"
32 echo "SLURM_JOB_PARTITION=${SLURM_JOB_PARTITION:-unknown}"
33 echo "SLURM_JOB_NUM_NODES=${SLURM_JOB_NUM_NODES:-unknown}"
34 echo "SLURM_NODELIST=${SLURM_NODELIST:-unknown}"
35 echo "CUDA_VISIBLE_DEVICES=${CUDA_VISIBLE_DEVICES:-unset}"
36 scontrol show job "${SLURM_JOB_ID}" || true
37
38 echo
39 echo "==== node config ====="
40 lscpu | sed -n '1,20p'
41 if [ -n "${SLURMD_NODENAME:-}" ]; then
42     scontrol show node "${SLURMD_NODENAME}" || true
43 fi
44 nvidia-smi -L || true
45 nvidia-smi || true
46
47 echo
48 echo "==== benchmark ====="
49 ./bin/linpack_cuda \
50     --start 1000 \
51     --step 500 \
52     --count 6 \
53     --eps 1e-6 \
54     --max-iters 15000 \
55     --threads 256 \
56     --repeat 3 \
57     --warmup 1 \
58     --csv "results/task1-cuda-${SLURM_JOB_ID}.csv"
```

Листинг 5: task\_1/scripts/run\_cuda.slurm

## Приложение В

```
1  #!/usr/bin/env bash
2  #SBATCH --job-name=task1-intel-linpack
3  #SBATCH --partition=tornado
4  #SBATCH --nodes=1
5  #SBATCH --ntasks=1
6  #SBATCH --cpus-per-task=56
7  #SBATCH --time=00:20:00
8  #SBATCH --output=stdio/%x-%j.out
9  #SBATCH --error=stdio/%x-%j.err
10
11  set -euo pipefail
12
13  TASK1_DIR="${SLURM_SUBMIT_DIR:-$PWD}"
14  module purge
15
16  LINPACK_DIR="${LINPACK_DIR:-$HOME/LINPACK}"
17  LINPACK_INPUT="${LINPACK_INPUT:-$TASK1_DIR/intel/lininput_report_xeon64}"
18
19  if [ ! -d "${LINPACK_DIR}" ]; then
20      echo "LINPACK directory not found: ${LINPACK_DIR}"
21      echo "Prepare Intel LINPACK in your home directory first."
22      echo "Example:"
23      echo "  mkdir -p \${HOME}/LINPACK"
24      echo "  tar -xzf \${HOME}/linpack.tgz -C \${HOME}/LINPACK"
25      exit 1
26  fi
27
28  resolve_linpack_dir() {
29      if [ -x "${LINPACK_DIR}/xlinpack_xeon64" ]; then
30          printf '%s\n' "${LINPACK_DIR}"
31          return 0
32      fi
33
34      local found
35      found="$(find "${LINPACK_DIR}" -name xlinpack_xeon64 2>/dev/null | head -n 1 || true)"
36      if [ -n "${found}" ]; then
37          dirname "${found}"
38          return 0
39      fi
40
41      return 1
42  }
43
44  if ! LINPACK_DIR="$(resolve_linpack_dir)"; then
45      echo "Intel LINPACK binary not found under: ${LINPACK_DIR}"
46      echo "Check archive contents with:"
47      echo "  find ${LINPACK_DIR} -name xlinpack_xeon64 2>/dev/null"
48      exit 1
49  fi
50
51  cd "${LINPACK_DIR}"
52
53  chmod +x ./ * || true
54  chmod -x ./ *.* || true
55  mkdir -p stdio
56
57  echo "==== account info ====="
58  whoami
59  hostname
60  date
61
62  echo
63  echo "==== slurm info ====="
64  echo "SLURM_JOB_ID=${SLURM_JOB_ID:-unknown}"
```

```

65 echo "SLURM_JOB_NAME=${SLURM_JOB_NAME:-unknown}"
66 echo "SLURM_JOB_PARTITION=${SLURM_JOB_PARTITION:-unknown}"
67 echo "SLURM_JOB_NUM_NODES=${SLURM_JOB_NUM_NODES:-unknown}"
68 echo "SLURM_NODELIST=${SLURM_NODELIST:-unknown}"
69 echo "OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-56}"
70 scontrol show job "${SLURM_JOB_ID}" || true
71
72 echo
73 echo "==== node config ====="
74 lscpu | sed -n '1,20p'
75 if [ -n "${SLURMD_NODENAME:-}" ]; then
76     scontrol show node "${SLURMD_NODENAME}" || true
77 fi
78
79 echo
80 echo "==== intel linpack ====="
81 echo "LINPACK_DIR=${LINPACK_DIR}"
82 echo "LINPACK_INPUT=${LINPACK_INPUT}"
83 export OMP_NUM_THREADS="${SLURM_CPUS_PER_TASK:-56}"
84 export MKL_NUM_THREADS="${SLURM_CPUS_PER_TASK:-56}"
85
86 srun ./xlinpack_xeon64 "${LINPACK_INPUT}"

```

Листинг 6: task\_1/scripts/run\_intel\_linpack.slurm

## Приложение Г

```
1 Shared-memory version of Intel(R) Distribution for LINPACK* Benchmark. *Other names and
  brands may be claimed as the property of others.
2 Custom data file for task1 report.
3 6 # number of tests
4 1000 1500 2000 2500 3000 3500 # problem sizes
5 1000 1504 2000 2504 3000 3504 # leading dimensions
6 8 6 6 5 5 5 # times to run a test
7 4 4 4 4 4 4 # alignment values (in KBytes)
```

Листинг 7: task\_1/intel/lininput\_report\_xeon64



## Приложение Д

```
1 #!/usr/bin/env bash
2  set -euo pipefail
3
4  ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
5  cd "$ROOT_DIR"
6
7  mkdir -p bin results
8
9  CUDA_ARCH="${CUDA_ARCH:-sm_35}"
10
11 module purge
12 module load compiler/gcc/11
13 module load nvidia/cuda/11.6u2
14
15 nvcc \
16     -ccbin g++ \
17     -O3 \
18     -std=c++14 \
19     -lineinfo \
20     -Wno-deprecated-gpu-targets \
21     -arch="${CUDA_ARCH}" \
22     -o bin/linpack_cuda \
23     src/main.cu
24
25 echo "Built: $ROOT_DIR/bin/linpack_cuda"
```

Листинг 8: task\_1/scripts/build.sh

## Приложение Е

```
1 #include <math.h>
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6
7 /*
8  * Задача: нахождение матрицы алгебраических дополнений.
9  *
10 * Дано: матрица A размером n x n, элементы в  $[-10^5, 10^5]$ ,  $2 \leq n \leq 100$ .
11 * Найти: матрицу B, где  $B[i][j] = (-1)^{(i+j)} * \det(M_{ij})$ ,
12 *       $M_{ij}$  – минор (матрица A без строки i и столбца j).
13 *
14 * Верификация:  $A * B^T = \det(A) * I$ .
15 *
16 * MPI-параллелизация: строки матрицы B распределяются между процессами.
17 */
18
19 static double determinant_lu(double *mat, int n) {
20     double det = 1.0;
21     for (int k = 0; k < n; k++) {
22         int pivot = k;
23         double max_val = fabs(mat[k * n + k]);
24         for (int r = k + 1; r < n; r++) {
25             double v = fabs(mat[r * n + k]);
26             if (v > max_val) { max_val = v; pivot = r; }
27         }
28         if (pivot != k) {
29             for (int c = 0; c < n; c++) {
30                 double tmp = mat[k * n + c];
31                 mat[k * n + c] = mat[pivot * n + c];
32                 mat[pivot * n + c] = tmp;
33             }
34             det = -det;
35         }
36         double diag = mat[k * n + k];
37         if (fabs(diag) < 1e-15) return 0.0;
38         det *= diag;
39         for (int r = k + 1; r < n; r++) {
40             double factor = mat[r * n + k] / diag;
41             for (int c = k + 1; c < n; c++)
42                 mat[r * n + c] -= factor * mat[k * n + c];
43         }
44     }
45     return det;
46 }
47
48 static void extract_minor(const double *A, double *sub, int n,
49                          int skip_row, int skip_col) {
50     int m = n - 1;
51     int sr = 0;
52     for (int r = 0; r < n; r++) {
53         if (r == skip_row) continue;
54         int sc = 0;
55         for (int c = 0; c < n; c++) {
56             if (c == skip_col) continue;
57             sub[sr * m + sc] = A[r * n + c];
58             sc++;
59         }
60         sr++;
61     }
62 }
63
64 static void generate_matrix(double *A, int n, unsigned int seed) {
65     srand(seed);
```

```

66     for (int i = 0; i < n * n; i++)
67         A[i] = ((double)rand() / RAND_MAX) * 2e5 - 1e5;
68 }
69
70 int main(int argc, char *argv[]) {
71     MPI_Init(&argc, &argv);
72
73     int rank, size;
74     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
75     MPI_Comm_size(MPI_COMM_WORLD, &size);
76
77     if (argc < 2) {
78         if (rank == 0)
79             fprintf(stderr,
80                 "Usage: mpirun -np <P> %s <n> [csv_file]\n", argv[0]);
81         MPI_Finalize();
82         return 1;
83     }
84
85     int n = atoi(argv[1]);
86     const char *csv_path = (argc >= 3) ? argv[2] : NULL;
87
88     if (n < 2 || n > 100) {
89         if (rank == 0)
90             fprintf(stderr, "Error: n must be in [2, 100], got %d\n", n);
91         MPI_Finalize();
92         return 1;
93     }
94
95     double *A = (double *)malloc((size_t)n * n * sizeof(double));
96
97     if (rank == 0)
98         generate_matrix(A, n, 42);
99
100    MPI_Bcast(A, n * n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
101
102    int base_rows = n / size;
103    int remainder = n % size;
104    int local_rows = base_rows + (rank < remainder ? 1 : 0);
105    int start_row = rank * base_rows + (rank < remainder ? rank : remainder);
106
107    double *local_B = (double *)malloc((size_t)local_rows * n * sizeof(double));
108    int m = n - 1;
109    double *sub = (double *)malloc((size_t)m * m * sizeof(double));
110
111    MPI_Barrier(MPI_COMM_WORLD);
112    double t_start = MPI_Wtime();
113
114    for (int li = 0; li < local_rows; li++) {
115        int i = start_row + li;
116        for (int j = 0; j < n; j++) {
117            extract_minor(A, sub, n, i, j);
118            double det = determinant_lu(sub, m);
119            double sign = ((i + j) % 2 == 0) ? 1.0 : -1.0;
120            local_B[li * n + j] = sign * det;
121        }
122    }
123
124    double t_end = MPI_Wtime();
125    double elapsed_ms = (t_end - t_start) * 1000.0;
126
127    double *B = NULL;
128    int *recvcunts = NULL, *displs = NULL;
129
130    if (rank == 0) {
131        B = (double *)malloc((size_t)n * n * sizeof(double));
132        recvcunts = (int *)malloc((size_t)size * sizeof(int));

```

```

133     displs      = (int *)malloc((size_t)size * sizeof(int));
134     int off = 0;
135     for (int r = 0; r < size; r++) {
136         int rr = base_rows + (r < remainder ? 1 : 0);
137         recvcunts[r] = rr * n;
138         displs[r] = off;
139         off += rr * n;
140     }
141 }
142
143 MPI_Gatherv(local_B, local_rows * n, MPI_DOUBLE,
144             B, recvcunts, displs, MPI_DOUBLE,
145             0, MPI_COMM_WORLD);
146
147 if (rank == 0) {
148     double *A_copy = (double *)malloc((size_t)n * n * sizeof(double));
149     memcpy(A_copy, A, (size_t)n * n * sizeof(double));
150     double det_A = determinant_lu(A_copy, n);
151     free(A_copy);
152
153     double max_err = 0.0;
154     for (int i = 0; i < n; i++) {
155         for (int j = 0; j < n; j++) {
156             double sum = 0.0;
157             for (int k = 0; k < n; k++)
158                 sum += A[i * n + k] * B[j * n + k];
159             double expected = (i == j) ? det_A : 0.0;
160             double err = fabs(sum - expected);
161             if (err > max_err) max_err = err;
162         }
163     }
164     double rel_err = (fabs(det_A) > 1e-15)
165                     ? max_err / fabs(det_A)
166                     : max_err;
167
168     printf("n=%d det(A)=%.6e verify_err=%.6e rel_err=%.6e "
169           "time=%.2f ms procs=%d\n",
170           n, det_A, max_err, rel_err, elapsed_ms, size);
171
172     if (n <= 5) {
173         printf("\nMatrix A:\n");
174         for (int i = 0; i < n; i++) {
175             for (int j = 0; j < n; j++)
176                 printf("%12.4f", A[i * n + j]);
177             printf("\n");
178         }
179         printf("\nCofactor matrix B:\n");
180         for (int i = 0; i < n; i++) {
181             for (int j = 0; j < n; j++)
182                 printf("%12.4f", B[i * n + j]);
183             printf("\n");
184         }
185     }
186
187     if (csv_path) {
188         FILE *fp = fopen(csv_path, "a");
189         if (fp) {
190             fprintf(fp, "%d,%d,%.4f,%.6e,%.6e\n",
191                     n, size, elapsed_ms, max_err, rel_err);
192             fclose(fp);
193         }
194     }
195
196     free(B);
197     free(recvcunts);
198     free(displs);
199 }

```

```
200
201     free(A);
202     free(local_B);
203     free(sub);
204
205     MPI_Finalize();
206     return 0;
207 }
```

task\_2/src/cofactor\_mpi.c

## Приложение Ж

```
1 #include <cuda_runtime.h>
2 #include <device_launch_parameters.h>
3 #include <math.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7
8 /*
9  * Задача: нахождение матрицы алгебраических дополнений (CUDA).
10  *
11  * Дано: матрица A размером n x n, элементы в  $[-10^5, 10^5]$ ,  $2 \leq n \leq 100$ .
12  * Найти:  $B[i][j] = (-1)^{(i+j)} * \det(M_{ij})$ .
13  *
14  * Каждый CUDA-поток вычисляет один элемент матрицы B:
15  *   1) извлекает минор (n-1) x (n-1) в рабочую область,
16  *   2) вычисляет определитель через LU-разложение с выбором ведущего элемента,
17  *   3) сохраняет результат с учётом знака.
18  *
19  * Для экономии памяти GPU элементы обрабатываются пакетами (batch).
20  */
21
22 #define CUDA_CHECK(call) do {                                     \
23     cudaError_t err = (call);                                     \
24     if (err != cudaSuccess) {                                     \
25         fprintf(stderr, "CUDA error at %s:%d: %s\n",             \
26             __FILE__, __LINE__, cudaGetErrorString(err));       \
27         exit(EXIT_FAILURE);                                       \
28     }                                                             \
29 } while (0)
30
31 #define MAX_N          100
32 #define DEF_THREADS    256
33
34 __device__ double dev_determinant(double *mat, int n) {
35     double det = 1.0;
36     for (int k = 0; k < n; k++) {
37         int pivot = k;
38         double mx = fabs(mat[k * n + k]);
39         for (int r = k + 1; r < n; r++) {
40             double v = fabs(mat[r * n + k]);
41             if (v > mx) { mx = v; pivot = r; }
42         }
43         if (pivot != k) {
44             for (int c = 0; c < n; c++) {
45                 double tmp = mat[k * n + c];
46                 mat[k * n + c] = mat[pivot * n + c];
47                 mat[pivot * n + c] = tmp;
48             }
49             det = -det;
50         }
51         double diag = mat[k * n + k];
52         if (fabs(diag) < 1e-15) return 0.0;
53         det *= diag;
54         for (int r = k + 1; r < n; r++) {
55             double f = mat[r * n + k] / diag;
56             for (int c = k + 1; c < n; c++)
57                 mat[r * n + c] -= f * mat[k * n + c];
58         }
59     }
60     return det;
61 }
62
63 __global__ void cofactor_kernel(const double *A, double *B,
64                                 double *workspace, int n,
65                                 int batch_start, int batch_count) {
```

```

66     int tid = blockIdx.x * blockDim.x + threadIdx.x;
67     if (tid >= batch_count) return;
68
69     int idx = batch_start + tid;
70     int i   = idx / n;
71     int j   = idx % n;
72     int m   = n - 1;
73
74     double *sub = workspace + (size_t)tid * m * m;
75
76     int sr = 0;
77     for (int r = 0; r < n; r++) {
78         if (r == i) continue;
79         int sc = 0;
80         for (int c = 0; c < n; c++) {
81             if (c == j) continue;
82             sub[sr * m + sc] = A[r * n + c];
83             sc++;
84         }
85         sr++;
86     }
87
88     double det = dev_determinant(sub, m);
89     double sign = ((i + j) % 2 == 0) ? 1.0 : -1.0;
90     B[i * n + j] = sign * det;
91 }
92
93 static void generate_matrix(double *A, int n, unsigned int seed) {
94     srand(seed);
95     for (int i = 0; i < n * n; i++)
96         A[i] = ((double)rand() / RAND_MAX) * 2e5 - 1e5;
97 }
98
99 static double host_determinant(double *mat, int n) {
100     double det = 1.0;
101     for (int k = 0; k < n; k++) {
102         int pivot = k;
103         double mx = fabs(mat[k * n + k]);
104         for (int r = k + 1; r < n; r++) {
105             double v = fabs(mat[r * n + k]);
106             if (v > mx) { mx = v; pivot = r; }
107         }
108         if (pivot != k) {
109             for (int c = 0; c < n; c++) {
110                 double tmp = mat[k * n + c];
111                 mat[k * n + c] = mat[pivot * n + c];
112                 mat[pivot * n + c] = tmp;
113             }
114             det = -det;
115         }
116         double diag = mat[k * n + k];
117         if (fabs(diag) < 1e-15) return 0.0;
118         det *= diag;
119         for (int r = k + 1; r < n; r++) {
120             double f = mat[r * n + k] / diag;
121             for (int c = k + 1; c < n; c++)
122                 mat[r * n + c] -= f * mat[k * n + c];
123         }
124     }
125     return det;
126 }
127
128 int main(int argc, char *argv[]) {
129     if (argc < 2) {
130         fprintf(stderr, "Usage: %s <n> [threads] [csv_file]\n", argv[0]);
131         return 1;
132     }

```

```

133
134     int n          = atoi(argv[1]);
135     int threads = (argc >= 3) ? atoi(argv[2]) : DEF_THREADS;
136     const char *csv_path = (argc >= 4) ? argv[3] : NULL;
137
138     if (n < 2 || n > MAX_N) {
139         fprintf(stderr, "Error: n must be in [2, %d], got %d\n", MAX_N, n);
140         return 1;
141     }
142
143     int total = n * n;
144     int m      = n - 1;
145     size_t sub_bytes = (size_t)m * m * sizeof(double);
146
147     double *A = (double *)malloc((size_t)n * n * sizeof(double));
148     double *B = (double *)malloc((size_t)n * n * sizeof(double));
149     generate_matrix(A, n, 42);
150
151     double *d_A, *d_B, *d_work;
152     CUDA_CHECK(cudaMalloc(&d_A, (size_t)n * n * sizeof(double)));
153     CUDA_CHECK(cudaMalloc(&d_B, (size_t)n * n * sizeof(double)));
154     CUDA_CHECK(cudaMemcpy(d_A, A, (size_t)n * n * sizeof(double),
155                          cudaMemcpyHostToDevice));
156
157     size_t free_mem, total_mem;
158     CUDA_CHECK(cudaMemGetInfo(&free_mem, &total_mem));
159     size_t reserved = (size_t)n * n * sizeof(double) * 2 + (1 << 20);
160     size_t avail = (free_mem > reserved) ? free_mem - reserved : sub_bytes;
161     int batch_size = (int)(avail / sub_bytes);
162     if (batch_size > total) batch_size = total;
163     if (batch_size < 1) batch_size = 1;
164
165     CUDA_CHECK(cudaMalloc(&d_work, (size_t)batch_size * sub_bytes));
166
167     cudaEvent_t t0, t1;
168     CUDA_CHECK(cudaEventCreate(&t0));
169     CUDA_CHECK(cudaEventCreate(&t1));
170     CUDA_CHECK(cudaEventRecord(t0));
171
172     for (int start = 0; start < total; start += batch_size) {
173         int count = batch_size;
174         if (start + count > total) count = total - start;
175         int blocks = (count + threads - 1) / threads;
176         cofactor_kernel<<<blocks, threads>>>(d_A, d_B, d_work,
177                                             n, start, count);
178         CUDA_CHECK(cudaGetLastError());
179     }
180
181     CUDA_CHECK(cudaEventRecord(t1));
182     CUDA_CHECK(cudaEventSynchronize(t1));
183     float elapsed_ms = 0;
184     CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, t0, t1));
185
186     CUDA_CHECK(cudaMemcpy(B, d_B, (size_t)n * n * sizeof(double),
187                          cudaMemcpyDeviceToHost));
188
189     /* ----- verification: A * B^T == det(A) * I ----- */
190     double *A_copy = (double *)malloc((size_t)n * n * sizeof(double));
191     memcpy(A_copy, A, (size_t)n * n * sizeof(double));
192     double det_A = host_determinant(A_copy, n);
193     free(A_copy);
194
195     double max_err = 0.0;
196     for (int i = 0; i < n; i++) {
197         for (int j = 0; j < n; j++) {
198             double sum = 0.0;
199             for (int k = 0; k < n; k++)

```



```

200         sum += A[i * n + k] * B[j * n + k];
201     double expected = (i == j) ? det_A : 0.0;
202     double err = fabs(sum - expected);
203     if (err > max_err) max_err = err;
204 }
205 }
206 double rel_err = (fabs(det_A) > 1e-15) ? max_err / fabs(det_A) : max_err;
207
208 printf("n=%d det(A)=%.6e verify_err=%.6e rel_err=%.6e "
209        "time=%.2f ms batch=%d\n",
210        n, det_A, max_err, rel_err, elapsed_ms, batch_size);
211
212 if (n <= 5) {
213     printf("\nMatrix A:\n");
214     for (int i = 0; i < n; i++) {
215         for (int j = 0; j < n; j++)
216             printf("%12.4f", A[i * n + j]);
217         printf("\n");
218     }
219     printf("\nCofactor matrix B:\n");
220     for (int i = 0; i < n; i++) {
221         for (int j = 0; j < n; j++)
222             printf("%12.4f", B[i * n + j]);
223         printf("\n");
224     }
225 }
226
227 if (csv_path) {
228     FILE *fp = fopen(csv_path, "a");
229     if (fp) {
230         fprintf(fp, "%d,cuda,%.4f,%.6e,%.6e\n",
231                n, elapsed_ms, max_err, rel_err);
232         fclose(fp);
233     }
234 }
235
236 free(A);
237 free(B);
238 CUDA_CHECK(cudaFree(d_A));
239 CUDA_CHECK(cudaFree(d_B));
240 CUDA_CHECK(cudaFree(d_work));
241 CUDA_CHECK(cudaEventDestroy(t0));
242 CUDA_CHECK(cudaEventDestroy(t1));
243 return 0;
244 }

```

task\_2/src/cofactor\_cuda.cu

## Приложение 3

```
1  #!/usr/bin/env bash
2  #SBATCH --job-name=task2-cofactor-mpi
3  #SBATCH --partition=tornado
4  #SBATCH --ntasks-per-node=1
5  #SBATCH --cpus-per-task=56
6  #SBATCH --time=00:30:00
7  #SBATCH --output=results/%x-%j.out
8  #SBATCH --error=results/%x-%j.err
9
10 set -euo pipefail
11
12 cd "${SLURM_SUBMIT_DIR}"
13
14 module purge
15 module load compiler/gcc/11
16 module load mpi/openmpi
17
18 mkdir -p results bin
19
20 ./scripts/build_mpi.sh
21
22 RANKS=${SLURM_JOB_NUM_NODES}
23
24 echo "==== account info ====="
25 whoami; hostname; date
26
27 echo
28 echo "==== slurm info ====="
29 echo "SLURM_JOB_ID=${SLURM_JOB_ID:-unknown}"
30 echo "SLURM_JOB_PARTITION=${SLURM_JOB_PARTITION:-unknown}"
31 echo "SLURM_JOB_NUM_NODES=${SLURM_JOB_NUM_NODES:-unknown}"
32 echo "SLURM_NODELIST=${SLURM_NODELIST:-unknown}"
33 echo "RANKS=${RANKS}"
34 scontrol show job "${SLURM_JOB_ID}" || true
35
36 echo
37 echo "==== node config ====="
38 lscpu | head -20
39 if [ -n "${SLURMD_NODENAME:-}" ]; then
40     scontrol show node "${SLURMD_NODENAME}" || true
41 fi
42
43 CSV="results/task2-mpi-${RANKS}n-${SLURM_JOB_ID}.csv"
44 echo "n,procs,time_ms,verify_err,rel_err" > "$CSV"
45
46 echo
47 echo "==== benchmark (${RANKS} nodes / ${RANKS} ranks) ====="
48 for N in 10 20 30 50 70 100; do
49     echo "--- n=$N ---"
50     mpirun -np "${RANKS}" --map-by ppr:1:node --bind-to none \
51         ./bin/cofactor_mpi "$N" "$CSV"
52 done
53
54 echo
55 echo "==== done ====="
```

Листинг 9: task\_2/scripts/run\_mpi.slurm

## Приложение И

```
1  #!/usr/bin/env bash
2  #SBATCH --job-name=task2-cofactor-cuda
3  #SBATCH --partition=tornado-k40
4  #SBATCH --nodes=1
5  #SBATCH --ntasks=1
6  #SBATCH --time=00:30:00
7  #SBATCH --output=results/%x-%j.out
8  #SBATCH --error=results/%x-%j.err
9
10 set -euo pipefail
11
12 cd "${SLURM_SUBMIT_DIR}"
13
14 module purge
15 module load compiler/gcc/11
16 module load nvidia/cuda/11.6u2
17
18 mkdir -p results bin
19
20 ./scripts/build_cuda.sh
21
22 echo "==== account info ====="
23 whoami; hostname; date
24
25 echo
26 echo "==== slurm info ====="
27 echo "SLURM_JOB_ID=${SLURM_JOB_ID:-unknown}"
28 echo "SLURM_JOB_PARTITION=${SLURM_JOB_PARTITION:-unknown}"
29 echo "SLURM_NODELIST=${SLURM_NODELIST:-unknown}"
30 scontrol show job "${SLURM_JOB_ID}" || true
31
32 echo
33 echo "==== node config ====="
34 lscpu | head -20
35 nvidia-smi -L || true
36 nvidia-smi || true
37
38 CSV="results/task2-cuda-${SLURM_JOB_ID}.csv"
39 echo "n,impl,time_ms,verify_err,rel_err" > "$CSV"
40
41 echo
42 echo "==== benchmark ====="
43 for N in 10 20 30 50 70 100; do
44     echo "--- n=$N ---"
45     ./bin/cofactor_cuda "$N" 256 "$CSV"
46 done
47
48 echo
49 echo "==== done ====="
```

Листинг 10: task\_2/scripts/run\_cuda.slurm

## Приложение Й

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3 cd "$(dirname "$0")/.."
4 mkdir -p bin
5 mpicc -O3 -std=c99 -lm -o bin/cofactor_mpi src/cofactor_mpi.c
6 echo "Built bin/cofactor_mpi"
```

Листинг 11: task\_2/scripts/build\_mpi.sh

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3 cd "$(dirname "$0")/.."
4 CUDA_ARCH="${CUDA_ARCH:-sm_35}"
5 mkdir -p bin
6 nvcc -ccbin g++ -O3 -arch="$CUDA_ARCH" -o bin/cofactor_cuda src/cofactor_cuda.cu
7 echo "Built bin/cofactor_cuda (arch=$CUDA_ARCH)"
```

Листинг 12: task\_2/scripts/build\_cuda.sh